
Relatório de Auditoria de Segurança — FrequenIA

Data: 17 de setembro de 2026

Escopo: branch desenvolvimento-mvp, incluindo backend Flask, consultas PostgreSQL/Supabase, templates e JavaScript web, app Flutter, migrations, Dockerfile, configurações e histórico Git.

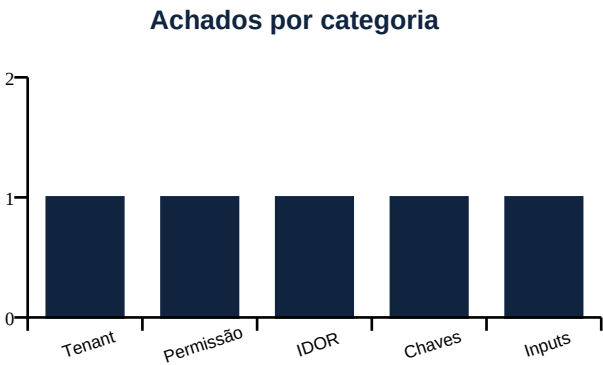
Nota metodológica: RLS foi mapeado ao controle efetivamente usado pela stack: filtros manuais com empresa_id derivado de g.auth_context. Gates visuais foram cruzados com decorators do Flask. Todos os handlers registrados foram enumerados para IDOR. Segredos foram buscados na árvore atual, artefatos de deploy, mobile, documentação e histórico Git. Sinks XSS e origens de dados foram rastreados até a renderização.

Auditoria estática orientada a evidência. Não foram realizadas ações destrutivas, exploração contra ambiente remoto ou leitura de segredos locais fora do repositório.

Resumo executivo

Foram confirmados cinco achados acionáveis: um crítico e quatro altos. O risco dominante está no módulo facial legado, que permanece registrado apesar da arquitetura moderna 1:1 já existente. A versão atual protege a maior parte das rotas por papel e tenant, mas as rotas legadas contornam esses controles. O histórico Git também contém um segredo de sessão real.

CRÍTICA	ALTA	MÉDIA	BAIXA
1	4	0	0



Prioridade imediata: desregistrar routes/face.py ou devolver 410 nas rotas legadas, rotacionar FLASK_SECRET_KEY e invalidar sessões. Em seguida, remover definitivamente a tabela/integração fotos baseada em nome.

Pontos fortes e pontos fracos

PONTOS FORTES	PONTOS FRACOS
• JWT HS256 exige claims sub, funcionario_id, sid, iat e exp; a sessão persistida é revalidada no banco (utils/auth_decorator.py:24-55, 95-125). • Segredos atuais obrigatórios falham no startup quando ausentes; CORS curinga é rejeitado e chave de teste é bloqueada em produção (app.py:15-21, 77-88). • Rotas modernas de biometria validam papel, tenant e funcionario_id, rejeitando identidade/decisão enviadas pelo cliente (routes/biometrics.py:62-85, 183-215, 370-390, 442-449). • Ocorrências pessoais e administrativas incluem empresa_id em listas, detalhes e transições; rotas pessoais também incluem funcionario_id e solicitante (routes/occurrences.py:244-260, 277-286, 326-349). • Controle de ponto e exportações derivam empresa da sessão e validam o funcionário na mesma empresa (routes/timekeeping.py:161-178, 287-349). • Sinks XSS dinâmicos relevantes usam textContent/Option.textContent ou escape explícito; não foi confirmado XSS explorável (static/js/ocorrencias.js:27-35; templates/gerenciarUsuario.html:35, 49-53). • .gitignore e .dockerignore excluem .env, chaves privadas, keystores e diretórios temporários.	• Blueprint facial legado público e mutável continua registrado. • Identidade legada é modelada por nome e busca biométrica global, incompatível com tenant isolation. • Caminhos de filesystem aceitam entrada bruta antes de rmtree/cv2.imwrite. • Segredo Flask histórico exige rotação mesmo após remoção do HEAD. • Não há RLS nas migrations; o banco depende integralmente da correção dos filtros da API. Isso não foi tratado como falha autônoma porque não há acesso direto do frontend ao Supabase, mas reduz defesa em profundidade.

Achados detalhados

Severidade	Arquivo:linha	Descrição
CRÍTICA	F-01 routes/face.py:43, 64-74, 170-185	Travessia de diretório permite exclusão recursiva sem autenticação O valor nome vem do JSON, é concatenado diretamente a temp_cadastros e o caminho resultante é entregue a shutil.rmtree. A rota /iniciar_cadastro não possui guarda de autenticação. Um nome como ../static sai da pasta temporária e pode apagar recursivamente um diretório acessível ao processo. A exploração exige apenas que o alvo exista e que o processo tenha permissão de escrita.
ALTA	F-02 app.py:114-120; routes/face.py:170-200, 256-257	Rotas legadas de cadastro facial estão registradas sem autorização O blueprint face é registrado no Flask, mas /iniciar_cadastro, /adicionar_foto e /finalizar_cadastro não usam require_roles nem access_token_required. A implementação moderna protege o cadastro facial como administrador, porém um cliente pode chamar as rotas legadas diretamente. Início e upload temporário são exploráveis sempre; a finalização também requer a tabela legada fotos e credenciais Cloudinary operacionais.
ALTA	F-03 routes/face.py:118-125, 396-421, 455-481	Reconhecimento facial legado faz busca global sem tenant O isolamento do projeto é manual: as queries modernas recebem empresa_id de g.auth_context. A query legada seleciona toda a tabela fotos, sem empresa_id, funcionario_id ou sessão, e a rota /reconhecer é pública. Com a tabela legada populada, uma imagem pode ser comparada contra pessoas de todas as empresas e a resposta revela nome e usuario_id.
ALTA	F-04 routes/face.py:173-185, 261-288, 333-345, 455-459	Nome controlado pelo cliente referencia e envenena identidade biométrica O fluxo legado trata nome como identificador direto do objeto. Não há vínculo com o usuário, funcionário ou empresa autenticados. Um chamador pode escolher o nome de outra pessoa, limpar seu staging, enviar imagens e, se o fluxo legado estiver operacional e houver menos de cinco registros, acrescentar embeddings associados ao mesmo nome. O reconhecimento ainda associa usuarios e fotos por igualdade de nome, ampliando colisões e envenenamento.
ALTA	F-05 app.py:9 no commit 8e0f6e89 (presente em e68467d8)	Segredo Flask foi commitado no histórico Git O histórico contém app.secret_key com valor literal de 23 caracteres. Embora a versão atual exija FLASK_SECRET_KEY no ambiente, remover o valor do HEAD não o remove dos commits. Se esse segredo foi usado ou reutilizado, qualquer pessoa com acesso ao repositório pode forjar cookies de sessão Flask; o backend aceita sessão como fallback em require_roles. O valor foi redigido neste relatório (SHA-256 prefixo 95B8D34B4DC6).

F-01 - Travessia de diretório permite exclusão recursiva sem autenticação

Categoria: Inputs sem tratamento **Severidade:** Crítica

Evidência: routes/face.py:43, 64-74, 170-185

```
PASTA_TEMP = "temp_cadastros"
def pasta_usuario(nome):
    return os.path.join(PASTA_TEMP, str(nome))
def limpar_pasta(caminho):
    if os.path.exists(caminho):
        shutil.rmtree(caminho, ignore_errors=True)
# /iniciar_cadastro: nome -> pasta_usuario -> limpar_pasta
```

Por que é explorável: O valor nome vem do JSON, é concatenado diretamente a temp_cadastros e o caminho resultante é entregue a shutil.rmtree. A rota /iniciar_cadastro não possui guarda de autenticação. Um nome como ../static sai da pasta temporária e pode apagar recursivamente um diretório acessível ao processo. A exploração exige apenas que o alvo exista e que o processo tenha permissão de escrita.

Correção recomendada: Desativar as rotas legadas. Se forem mantidas, usar identificador UUID derivado da sessão, resolver o caminho e exigir que ele permaneça sob uma raiz dedicada; nunca aceitar segmentos de caminho do cliente.

F-02 - Rotas legadas de cadastro facial estão registradas sem autorização

Categoria: Permissão no navegador **Severidade:** Alta

Evidência: app.py:114-120; routes/face.py:170-200, 256-257

```
app.register_blueprint(face_bp)
@face_bp.route("/iniciar_cadastro", methods=["POST"])
def iniciar_cadastro(): ...
@face_bp.route("/adicionar_foto", methods=["POST"])
def adicionar_foto(): ...
@face_bp.route("/finalizar_cadastro", methods=["POST"])
def finalizar_cadastro(): ...
```

Por que é explorável: O blueprint face é registrado no Flask, mas /iniciar_cadastro, /adicionar_foto e /finalizar_cadastro não usam require_roles nem access_token_required. A implementação moderna protege o cadastro facial como administrador, porém um cliente pode chamar as rotas legadas diretamente. Início e upload temporário são exploráveis sempre; a finalização também requer a tabela legada fotos e credenciais Cloudinary operacionais.

Correção recomendada: Remover o registro das rotas legadas ou responder 410. Se houver migração temporária, aplicar require_roles('administrador'), identidade por funcionario_id, limites de upload e o mesmo serviço 1:1 usado por routes/biometrics.py.

F-03 - Reconhecimento facial legado faz busca global sem tenant

Categoria: Banco sem tranca **Severidade:** Alta

Evidência: routes/face.py:118-125, 396-421, 455-481

```
SELECT nome, embedding <=> %s::vector AS distancia
FROM fotos
ORDER BY distancia
LIMIT 5
# resposta inclui nome e usuario_id
```

Por que é explorável: O isolamento do projeto é manual: as queries modernas recebem empresa_id de g.auth_context. A query legada seleciona toda a tabela fotos, sem empresa_id, funcionario_id ou sessão, e a rota /reconhecer é pública. Com a tabela legada populada, uma imagem pode ser comparada contra pessoas de todas as empresas e a resposta revela nome e usuario_id.

Correção recomendada: Eliminar a identificação global e usar apenas verificação 1:1 do vínculo autenticado. Remover a tabela/rota legada e manter empresa_id + funcionario_id em todas as consultas biométricas.

F-04 - Nome controlado pelo cliente referencia e envenena identidade biométrica

Categoria: IDOR **Severidade:** Alta

Evidência: routes/face.py:173-185, 261-288, 333-345, 455-459

```
nome = dados.get("nome")
pasta = pasta_usuario(nome)
SELECT COUNT(*) FROM fotos WHERE nome = %s
INSERT INTO fotos (nome, url, embedding) VALUES (%s, %s, %s)
INNER JOIN fotos f ON u.nome = f.nome
```

Por que é explorável: O fluxo legado trata nome como identificador direto do objeto. Não há vínculo com o usuário, funcionário ou empresa autenticados. Um chamador pode escolher o nome de outra pessoa, limpar seu staging, enviar imagens e, se o fluxo legado estiver operacional e houver menos de cinco registros, acrescentar embeddings associados ao mesmo nome. O reconhecimento ainda associa usuarios e fotos por igualdade de nome, ampliando colisões e envenenamento.

Correção recomendada: Não aceitar nome como autoridade. Receber funcionario_id apenas em rota administrativa, validar a posse no tenant e persistir por chave composta empresa_id/funcionario_id. Tornar o recadastro transacional e revogar o template anterior.

F-05 - Segredo Flask foi commitado no histórico Git

Categoria: Chaves expostas **Severidade:** Alta

Evidência: app.py:9 no commit 8e0f6e89 (presente em e68467d8)

```
app.secret_key = "[SEGREDO REDIGIDO; 23 caracteres]"
```

Por que é explorável: O histórico contém app.secret_key com valor literal de 23 caracteres. Embora a versão atual exija FLASK_SECRET_KEY no ambiente, remover o valor do HEAD não o remove dos commits. Se esse segredo foi usado ou reutilizado, qualquer pessoa com acesso ao repositório pode forjar cookies de sessão Flask; o backend aceita sessão como fallback em require_roles. O valor foi redigido neste relatório (SHA-256 prefixo 95B8D34B4DC6).

Correção recomendada: Rotacionar FLASK_SECRET_KEY em todos os ambientes, invalidar sessões, verificar reutilização e reescrever o histórico se a política do repositório permitir. Adicionar secret scanning em CI e proteção pre-commit.

Cobertura e pontos verificados como corretos

Área	Cobertura e resultado
Stack	Python 3.10, Flask 3, psycopg2/PostgreSQL (schema Supabase), JWT PyJWT + auth_sessions, Jinja/JavaScript, Flutter/Dart, Dockerfile. Sem ORM/query builder; SQL parametrizado manual. Sem CI, Helm ou Terraform versionados.
Rotas	73 handlers e 78 decorators de rota enumerados em app.py e routes/*.py. 52 handlers possuem guarda explícita; os demais foram classificados como públicos intencionais (health, páginas, login/reset/hora) ou legado facial. Todos os handlers com IDs em path/body/query foram revisados.
Tenant/IDOR	Biometria moderna, ocorrências, turnos, jornadas, marcações, gestão de usuários e exportações foram cruzadas com empresa_id da sessão. Os handlers modernos por UUID retornam 404 quando o objeto não pertence ao tenant. A exceção acionável é routes/face.py.
Gates de papel	Menu e páginas administrativas foram cruzados com cadastrar_usuario, atualizar_usuario, status, jornadas, biometria, controle de ponto, exportação e ocorrências. O backend moderno repete os gates. As rotas legadas não repetem.
Segredos	Árvore atual, .env.example, Dockerfile, migrations/scripts, documentação, Flutter e histórico Git pesquisados. Nenhuma chave real foi confirmada no HEAD ou em bundle versionado; um segredo Flask foi confirmado no histórico. Defaults funcionais de porta/timeout não são segredos.
XSS	Pesquisados innerHTML/outerHTML/insertAdjacentHTML, renderização HTML/Markdown, eval/new Function, URLs controladas e templates backend. Não há biblioteca de sanitização instalada; nos pontos atuais ela não é necessária porque os dados dinâmicos são renderizados como texto/autoescaped. O innerHTML de gerenciarEmpresa é inseguro em abstrato, mas a API atual não fornece os campos interpolados e a página está fora do menu, portanto não foi reportado como achado explorável.
Testes	A auditoria foi estática. Tentativas de executar testes unitários falharam antes da coleta porque pytest/Flask não estão instalados nos runtimes Python disponíveis. Nenhuma dependência foi instalada globalmente. A geração do PDF usa o runtime isolado fornecido pelo Codex.

Recomendações priorizadas

Prioridade	Ação
P1	Desregistrar ou devolver 410 para todas as rotas mutáveis e de reconhecimento de routes/face.py.
P1	Rotacionar FLASK_SECRET_KEY em todos os ambientes e invalidar sessões emitidas com o segredo histórico.
P1	Eliminar o uso de nome em paths e como chave biométrica; usar empresa_id + funcionario_id validados.
P2	Remover a tabela fotos/Cloudinary legados após migração controlada para biometrias e verificação 1:1.
P2	Adicionar testes negativos de rota anônima, outro papel, outro tenant, path traversal e homônimos.
P2	Adicionar secret scanning no CI e pre-commit; decidir sobre purge do histórico e forks.
P3	Criar papel PostgreSQL de mínimo privilégio e avaliar RLS como defesa em profundidade, sem substituir os filtros da API.
P3	Adicionar CSP e manter política de evitar innerHTML com dados dinâmicos, ainda que não haja XSS confirmado agora.

ISSUES PARA O GITHUB

Textos completos em Markdown, prontos para copiar e colar. Evidências sensíveis permanecem redigidas.

```
--- ISSUE 1 ---
# [Segurança] Bloquear travessia de diretório no cadastro facial legado

**Labels sugeridas:** security, severidade:critica

## Descrição
O valor nome vem do JSON, é concatenado diretamente a temp_cadastros e o caminho resultante é entregue a shutil.rmtree. A rota /iniciar_cadastro não possui guarda de autenticação. Um nome como ../static sai da pasta temporária e pode apagar recursivamente um diretório acessível ao processo. A exploração exige apenas que o alvo exista e que o processo tenha permissão de escrita.

## Evidência
`routes/face.py:43, 64-74, 170-185`

```text
PASTA_TEMP = "temp_cadastros"
def pasta_usuario(nome):
 return os.path.join(PASTA_TEMP, str(nome))
def limpar_pasta(caminho):
 if os.path.exists(caminho):
 shutil.rmtree(caminho, ignore_errors=True)
/iniciar_cadastro: nome -> pasta_usuario -> limpar_pasta
```

## Impacto
Exclusão recursiva de diretórios graváveis pelo processo, indisponibilidade e perda de arquivos da aplicação.

## Sugestão de correção
Desativar as rotas legadas. Se forem mantidas, usar identificador UUID derivado da sessão, resolver o caminho e exigir que ele permaneça sob uma raiz dedicada; nunca aceitar segmentos de caminho do cliente.

## Critérios de aceite
- [ ] As rotas legadas foram removidas/desativadas ou exigem autenticação e autorização explícitas.
- [ ] Nenhum valor do cliente participa de um caminho de arquivo sem normalização e allowlist.
- [ ] O caminho resolvido é rejeitado quando não está contido na raiz temporária dedicada.
- [ ] Teste automatizado cobre ../, ..\ e caminhos absolutos sem tocar arquivos fora da raiz.
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
# [Segurança] Remover ou proteger endpoints legados de cadastro facial

**Labels sugeridas:** security, severidade:alta

## Descrição
O blueprint face é registrado no Flask, mas /iniciar_cadastro, /adicionar_foto e /finalizar_cadastro não usam require_roles nem access_token_required. A implementação moderna protege o cadastro facial como administrador, porém um cliente pode chamar as rotas legadas diretamente. Início e upload temporário são exploráveis sempre; a finalização também requer a tabela legada fotos e credenciais Cloudinary operacionais.

## Evidência
`app.py:114-120; routes/face.py:170-200, 256-257`

```text
app.register_blueprint(face_bp)
@face_bp.route("/iniciar_cadastro", methods=["POST"])
def iniciar_cadastro(): ...
@face_bp.route("/adicionar_foto", methods=["POST"])
def adicionar_foto(): ...
@face_bp.route("/finalizar_cadastro", methods=["POST"])
def finalizar_cadastro(): ...
```

## Impacto
Uso não autorizado de CPU, disco e Cloudinary, além de alteração indevida de dados biométricos legados.

## Sugestão de correção
Remover o registro das rotas legadas ou responder 410. Se houver migração temporária, aplicar require_roles('administrador'), identidade por funcionario_id, limites de upload e o mesmo serviço 1:1 usado por routes/biometrics.py.

## Critérios de aceite
- [ ] /iniciar_cadastro, /adicionar_foto e /finalizar_cadastro retornam 404/410 ou exigem administrador autenticado.
- [ ] O backend deriva empresa e ator da sessão, sem confiar em identidade enviada pelo navegador.
- [ ] Há testes negativos para anônimo, funcionário, gestor e tenant diferente.
- [ ] O blueprint legado não expõe rota mutável sem uma guarda equivalente no servidor.

--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
# [Segurança] Eliminar busca facial global sem isolamento por tenant

**Labels sugeridas:** security, severidade:alta

## Descrição
O isolamento do projeto é manual: as queries modernas recebem empresa_id de g.auth_context. A query legada seleciona toda a tabela fotos, sem empresa_id, funcionario_id ou sessão, e a rota /reconhecer é pública. Com a tabela legada populada, uma imagem pode ser comparada contra pessoas de todas as empresas e a resposta revela nome e usuario_id.

## Evidência
`routes/face.py:118-125, 396-421, 455-481`

```text
SELECT nome, embedding <=> %s::vector AS distancia
FROM fotos
ORDER BY distancia
LIMIT 5
resposta inclui nome e usuario_id
```

## Impacto
Vazamento de identidade entre empresas e comparação biométrica global fora do vínculo autenticado.

## Sugestão de correção
Eliminar a identificação global e usar apenas verificação 1:1 do vínculo autenticado. Remover a tabela/rota legada e manter empresa_id + funcionario_id em todas as consultas biométricas.

## Critérios de aceite
- [ ] Nenhuma query facial percorre pessoas fora do empresa_id autenticado.
- [ ] O fluxo de produção usa verificação 1:1 por funcionario_id da sessão.
- [ ] A resposta não revela nome, usuario_id ou vizinho biométrico de outro tenant.
- [ ] Teste de integração prova que imagens e IDs de outra empresa não produzem diferença observável.
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
# [Segurança] Substituir nome por identidade autorizada no cadastro biométrico

**Labels sugeridas:** security, severidade:alta

## Descrição
O fluxo legado trata nome como identificador direto do objeto. Não há vínculo com o usuário,
funcionário ou empresa autenticados. Um chamador pode escolher o nome de outra pessoa, limpar seu
staging, enviar imagens e, se o fluxo legado estiver operacional e houver menos de cinco registros,
acrescentar embeddings associados ao mesmo nome. O reconhecimento ainda associa usuarios e fotos por
igualdade de nome, ampliando colisões e envenenamento.

## Evidência
`routes/face.py:173-185, 261-288, 333-345, 455-459`

```text
nome = dados.get("nome")
pasta = pasta_usuario(nome)
SELECT COUNT(*) FROM fotos WHERE nome = %s
INSERT INTO fotos (nome, url, embedding) VALUES (%s, %s, %s)
INNER JOIN fotos f ON u.nome = f.nome
```

## Impacto
Envenenamento de templates biométricos, colisão entre homônimos e associação a usuário incorreto.

## Sugestão de correção
Não aceitar nome como autoridade. Receber funcionario_id apenas em rota administrativa, validar a
posse no tenant e persistir por chave composta empresa_id/funcionario_id. Tornar o recadastro
transacional e revogar o template anterior.

## Critérios de aceite
- [ ] O cadastro usa funcionario_id validado no empresa_id da sessão, nunca nome como chave.
- [ ] Homônimos não compartilham arquivos, embeddings ou registros.
- [ ] Recadastro bloqueia e substitui o template correto em transação.
- [ ] Testes cobrem ID de outro tenant, nome duplicado e tentativa anônima.
--- FIM ISSUE 4 ---
```

```
--- ISSUE 5 ---
# [Segurança] Rotacionar segredo Flask exposto no histórico Git

**Labels sugeridas:** security, severidade:alta

## Descrição
O histórico contém app.secret_key com valor literal de 23 caracteres. Embora a versão atual exija
FLASK_SECRET_KEY no ambiente, remover o valor do HEAD não o remove dos commits. Se esse segredo foi
usado ou reutilizado, qualquer pessoa com acesso ao repositório pode forjar cookies de sessão Flask; o
backend aceita sessão como fallback em require_roles. O valor foi redigido neste relatório (SHA-256
prefixo 95B8D34B4DC6).

## Evidência
`app.py:9 no commit 8e0f6e89 (presente em e68467d8)`

```text
app.secret_key = "[SEGREDO REDIGIDO; 23 caracteres]"
```

## Impacto
Forja de cookies de sessão e possível elevação de privilégios caso o segredo histórico ainda seja
válido ou reutilizado.

## Sugestão de correção
Rotacionar FLASK_SECRET_KEY em todos os ambientes, invalidar sessões, verificar reutilização e
reescrever o histórico se a política do repositório permitir. Adicionar secret scanning em CI e
proteção pre-commit.

## Critérios de aceite
- [ ] FLASK_SECRET_KEY foi rotacionada em todos os ambientes e sessões antigas foram invalidadas.
- [ ] Foi confirmado que o valor não é reutilizado em outros serviços.
- [ ] Secret scanning bloqueia novos commits com segredos reais.
- [ ] A equipe decidiu e documentou se fará purge do histórico e rotação de forks/caches.
--- FIM ISSUE 5 ---
```